

censtrunc

Цензурированная и усечённая нормальная регрессия с произвольными порогами

Автор: Рамис Пайгин

Пакет: `censtrunc` v0.1.0

Аннотация

`censtrunc` — Python-пакет для оценивания методом максимального правдоподобия двух семейств моделей с ограниченной зависимой переменной: (i) **цензурированной и усечённой нормальной регрессии** с произвольными левым и правым порогами, обобщающей классический Tobit (Tobin, 1958), и (ii) **модели селективного отбора Хекмана** (Heckman, 1979) в двух вариантах — двухшаговом и совместной MLE. Пакет предоставляет программный интерфейс в стиле `scikit-learn`, итоговую таблицу результатов в стиле `statsmodels`, предельные эффекты через `get_margeff`, тест отношения правдоподобия с гипотезой в виде строки и поддержку формул в стиле `patsy`. Для цензурированного случая оценивание ведётся в глобально вогнутой параметризации Олсена (1978).

Отчёт построен по двум семействам моделей. Каждая часть содержит три раздела: **теория**, **примеры применения API** на симулированных и реальных данных, **эмпирическая проверка корректности** — аналитические предельные случаи и кросс-валидация против независимых реализаций.

1. Введение

Обычная линейная регрессия предполагает, что зависимая переменная наблюдается на всей вещественной оси. На практике она часто ограничена:

- **Цензурирование:** значения вне интервала $[L, R]$ заменяются ближайшим порогом. Типичные примеры — оценка удовлетворённости по шкале 0–10 или зарплата с институциональным потолком.
- **Усечение:** значения вне $[L, R]$ полностью отсутствуют в выборке. Так устроены опросные данные, где регистрируются только респонденты с доходом выше определённого уровня, или административные данные о сделках выше порога обязательной отчётности.

- **Селективный отбор:** переменная наблюдается лишь тогда, когда срабатывает отдельное *уравнение отбора*. Зарплата фиксируется только у занятых; объём потребления — только у согласившихся участвовать в опросе.

Метод наименьших квадратов даёт смещённые оценки во всех трёх случаях. Классические решения: **Tobit** (Tobin, 1958) для односторонней цензуры в нуле, **усечённая нормальная регрессия** для одностороннего усечения и **модель Хекмана** (Heckman, 1979) — для третьего случая.

`censtrunc` объединяет все три модели в одной Python-библиотеке с общим API и единой поверхностью для прогнозов и интерпретации (`predict` , `get_margeff` , `lr_test` , `from_formula`). Разделы 3 и 4 подробно описывают оба семейства моделей.

2. Что уже реализовано в Python

Если для R эти модели реализованы давно и удобно — пакет `survival::survreg` (на нём построен `AER::tobit`) покрывает цензурированный случай, `truncreg::truncreg` — усечённый, а `sampleSelection::selection` — обе версии оценки Хекмана, — то в Python ситуация заметно более разрозненная.

Пакет	Цензурирование / Tobit	Усечение	Хекит
statsmodels	публичного Tobit-класса нет	в публичном API отсутствует	не реализован
scikit-learn	—	—	—
linearmodels	—	—	—
lifelines	левая цензура в Вейбулле и логнормальной AFT	—	—

Пакет	Цензурирование / Tobit	Усечение	Хекит
scikit-survival	правая цензура, AFT-модели	—	—
py4etrics	Tobit с произвольными L, R	Truncreg с произвольными L, R	двухшаговый Heckit (значение <code>method='mle'</code> молча игнорируется)
stnwanekezie/TobitRegression	один класс Tobit поверх <code>statsmodels.OLS</code>	—	—
PyMC / bambi	байесовский вариант через Censored	байесовский вариант через Truncated	можно собрать вручную
marginaleffects	—	—	—

Что нового добавляет `censtrunc`

- **Двустороннее цензурирование и усечение** с произвольными L и R во всех трёх оценщиках (односторонний случай получается как частный).
- **Совместная MLE для модели Хекмана**, а не только двухшаговый алгоритм; для двухшагового предусмотрена парная бутстрап-процедура.
- **Предельные эффекты** через единый `get_margeff` в стиле `statsmodels` для всех трёх моделей. Цензурированные модели дополнительно отдадут три эффекта на вероятности зон; Хекит — четыре (`latent` , `conditional` , `unconditional` , `prob-selected`).
- **Тест отношения правдоподобия** в двух формах: между парой подогнанных моделей либо `model.lr_test(hypotheses)` со строкой гипотез (`'(x1 = 0), (x2 = x3)', '2*x1 + x4 = 1'`).
- **Формулы в стиле patsy** во всех трёх классах.
- Единый `predict()` возвращает любое подмножество из шести величин через однобуквенную строку `kind` — для цензурированной модели это `'hctlmr'` , для модели Хекмана — `'snpohu'` .
- **Кросс-валидация против внешних реализаций** в тестах — см. §3.3 и §4.3.

Часть А. Цензурированная и усечённая нормальная регрессия

3.1 Теория

Латентная регрессия

Обе модели исходят из одной и той же латентной линейной спецификации с нормальной ошибкой:

$$Y^* = X'\beta + e, \quad e | X \sim \mathcal{N}(0, \sigma^2).$$

Различие — лишь в правиле, по которому латентная величина Y^* переходит в наблюдаемые данные.

Цензурированная модель

Значения за пределами $[L, R]$ **подменяются** ближайшим порогом:

$$Y = \min(R, \max(L, Y^*)).$$

Обозначая $\alpha_L = (L - X'\beta)/\sigma$ и $\alpha_R = (R - X'\beta)/\sigma$, в логарифме правдоподобия складываются точечные массы на порогах и нормальная плотность на интервале:

$$\ell(\beta, \sigma) = \sum_{Y_i=L} \log \Phi(\alpha_{L,i}) + \sum_{Y_i=R} \log [1 - \Phi(\alpha_{R,i})] + \sum_{L < Y_i < R} [\log \phi(\alpha_i) - \log \sigma],$$

где ϕ и Φ — плотность и функция распределения стандартного нормального закона. Классический Tobit соответствует частному случаю $L = 0, R = +\infty$.

Параметризация Олсена

Оптимизация ведётся в переменных $(\gamma, \nu) = (\beta/\sigma, 1/\sigma)$. Олсен (1978) показал, что в этих координатах логарифм правдоподобия **глобально вогнут**, поэтому любой градиентный метод сходится к единственному максимуму. После сходимости пакет совершает обратное преобразование к (β, σ) и вычисляет стандартные ошибки по наблюдаемой информационной матрице.

Усечённая модель

В усечённой выборке наблюдений вне (L, R) **нет вовсе**. Плотность — это нормальная плотность, перенормированная на вероятность попадания в интервал:

$$f(y_i \mid L < Y_i^* < R) = \frac{\sigma^{-1} \phi((y_i - X_i' \beta) / \sigma)}{\Phi(\alpha_{R,i}) - \Phi(\alpha_{L,i})}.$$

Условные средние и предельные эффекты

Поддерживаются три условных средних (Hansen, 2022, гл. 27):

- **Латентное:** $\mathbb{E}[Y^* \mid X] = X' \beta$
- **Цензурированное:** $\mathbb{E}[Y \mid X]$ с учётом точечных масс
- **Усечённое:** $\mathbb{E}[Y \mid X, L < Y < R]$

Предельные эффекты доступны в форме **AME** (усреднение по выборке) и **MEM** (в точке средних регрессоров), обе со стандартными ошибками по дельта-методу.

Цензурированный предельный эффект на $\mathbb{E}[Y \mid X]$ имеет вид

$\beta_j \cdot [\Phi(\alpha_R) - \Phi(\alpha_L)]$ — латентный наклон, умноженный на вероятность оказаться внутри интервала.

Статистический вывод

Каждая подогнанная модель содержит общий LR-тест против модели только с константой и псевдо- R^2 Мак-Фаддена. Сравнить произвольные вложенные модели позволяет `lr_test`, возвращающий $LR = 2(\ell_{\text{full}} - \ell_{\text{restricted}}) \sim \chi_q^2$.

3.2 Применение

Четыре подробно разобранных примера показывают API на постепенно усложняющихся данных:

- 3.2.1 двусторонняя цензура на симулированных данных,
- 3.2.2 классический Tobit с левой цензурой на данных Fair (1978) об измене,
- 3.2.3 усечённая регрессия на симулированных данных,
- 3.2.4 параллельная иллюстрация усечения и цензуры.

3.2.1 Двусторонняя цензурированная регрессия

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from censtrunc import CensoredRegression, lr_test

rng = np.random.default_rng(2026)
n = 3000
```

Процесс генерации данных

$$Y_i^* = 1.0 + 0.5 X_{1i} - 0.3 X_{2i} + 0.2 X_{3i} + e_i, \quad e_i \sim \mathcal{N}(0, 1).$$

Наблюдается ограниченная величина: значения ниже $L = 0$ и выше $R = 2.5$ подменяются ближайшим порогом. Это типичная картина двустороннего цензурирования — например, балл удовлетворённости по шкале 0–2.5.

```
In [2]: X = rng.normal(size=(n, 3))
beta_true = np.array([1.0, 0.5, -0.3, 0.2])
sigma_true = 1.0
y_star = beta_true[0] + X @ beta_true[1:] + rng.normal(scale=sigma_true, size=n)
L, R = 0.0, 2.5
y = np.clip(y_star, L, R)

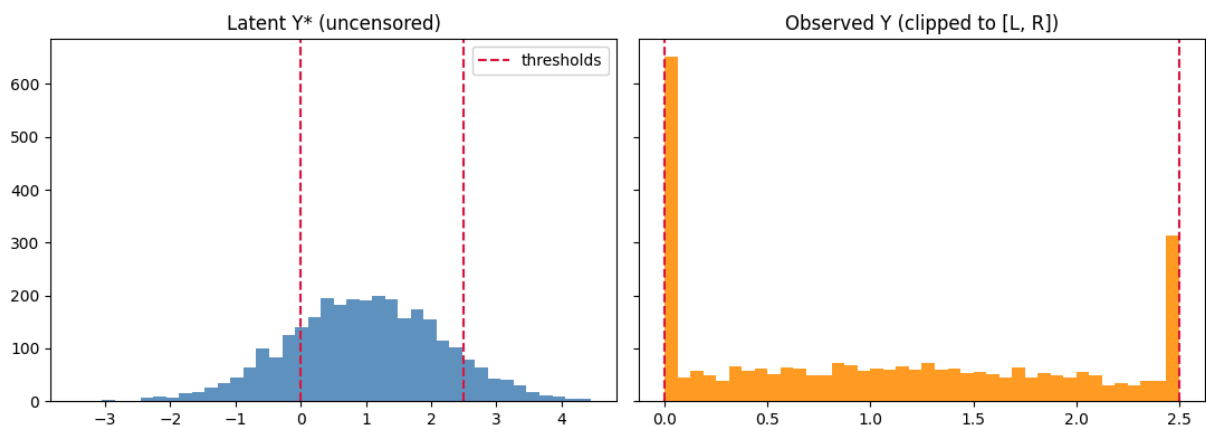
print(f'Left-censored:  {np.mean(y == L):.1%}')
print(f'Right-censored: {np.mean(y == R):.1%}')
print(f'Interior:       {np.mean((y > L) & (y < R)):.1%}')

Left-censored:  20.0%
Right-censored:  9.8%
Interior:       70.2%
```

Распределения латентной и наблюдаемой переменных

Скопления на L и R в распределении наблюдаемой величины — характерный признак двусторонней цензуры.

```
In [3]: fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
axes[0].hist(y_star, bins=40, color='steelblue', alpha=0.85)
axes[0].set_title('Latent Y* (uncensored)')
axes[0].axvline(L, color='crimson', linestyle='--', label='thresholds')
axes[0].axvline(R, color='crimson', linestyle='--')
axes[0].legend()
axes[1].hist(y, bins=40, color='darkorange', alpha=0.85)
axes[1].set_title('Observed Y (clipped to [L, R])')
axes[1].axvline(L, color='crimson', linestyle='--')
axes[1].axvline(R, color='crimson', linestyle='--')
plt.tight_layout(); plt.show()
```



Оценивание цензурированной регрессии

По умолчанию пакет использует параметризацию Олсена: в ней логарифм правдоподобия глобально вогнут, что гарантирует сходимость любого градиентного оптимизатора (Olsen, 1978).

```
In [4]: model = CensoredRegression(left=L, right=R).fit(
        X, y, feature_names=['x1', 'x2', 'x3'])
print(model.summary())
```

```
=====
                        Censored Regression Results
=====
```

Dep. Variable:	y	No. Observations:	3000
Model:	CensoredRegression	Df Model:	4
Method:	MLE	Df Residuals:	2995
Date:	Wed, 03 Jun 2026	Log-Likelihood:	-3831.9928
Time:	02:06:06	LL-Null:	-4286.2006
AIC:	7673.9855	LLR p-value:	1.323e-196
BIC:	7704.0174	Pseudo R-squ.:	0.1060
Left threshold:	0	Right threshold:	2.5

```
=====
```

	coef	std err	z	P> z	[0.025	0.975]
sigma	0.9959	0.0165	60.3216	0.0000	0.9635	1.0282
const	0.9963	0.0190	52.4873	0.0000	0.9591	1.0336
x1	0.5126	0.0195	26.2589	0.0000	0.4744	0.5509
x2	-0.2565	0.0188	-13.6315	0.0000	-0.2934	-0.2196
x3	0.2030	0.0192	10.5893	0.0000	0.1655	0.2406

```
=====
```

Left-censored observations:	601 (20.03%)
Right-censored observations:	293 (9.77%)
Uncensored observations:	2106 (70.20%)
Optimiser converged:	True

```
=====
```

Сравнение с OLS

На цензурированных данных OLS даёт смещённые оценки (Greene, 1981): наклоны сжимаются примерно пропорционально вероятности цензурирования. Tobit-оценка по методу максимального правдоподобия в нашей реализации состоятельна.

```
In [5]: X_with_int = np.column_stack([np.ones(n), X])
ols_beta, *_ = np.linalg.lstsq(X_with_int, y, rcond=None)

comparison = pd.DataFrame({
    'true': beta_true,
    'OLS (biased)': ols_beta,
    'Censored MLE': model.coef_,
}, index=['const', 'x1', 'x2', 'x3'])
comparison['OLS bias'] = comparison['OLS (biased)'] - comparison['true']
comparison['MLE bias'] = comparison['Censored MLE'] - comparison['true']
comparison.round(4)
```

Out [5]:

	true	OLS (biased)	Censored MLE	OLS bias	MLE bias
const	1.0	1.0706	0.9963	0.0706	-0.0037
x1	0.5	0.3648	0.5126	-0.1352	0.0126
x2	-0.3	-0.1794	-0.2565	0.1206	0.0435
x3	0.2	0.1460	0.2030	-0.0540	0.0030

Прогнозы: шесть величин одной командой

`model.predict(X)` возвращает DataFrame со всеми шестью величинами — тремя условными средними и тремя вероятностями зон. У каждой колонки — однобуквенная мнемоника:

- **h** — *hidden*, латентное среднее $\mathbb{E}[Y^* | X] = X'\beta$
- **c** — *censored*, среднее $\mathbb{E}[Y | X]$
- **t** — *truncated*, среднее $\mathbb{E}[Y | X, L < Y < R]$
- **l** — вероятность левой зоны $P(Y = L | X) = \Phi(\alpha_L)$
- **m** — вероятность интервала $P(L < Y < R | X) = \Phi(\alpha_R) - \Phi(\alpha_L)$
- **r** — вероятность правой зоны $P(Y = R | X) = 1 - \Phi(\alpha_R)$

In [6]:

```
X_show = X[:6]
model.predict(X_show).round(3) # default = all six columns
```

Out [6]:

	latent	censored	truncated	prob_left	prob_interior	prob_right
0	0.143	0.470	0.816	0.443	0.548	0.009
1	1.489	1.438	1.351	0.067	0.778	0.155
2	0.704	0.831	1.023	0.240	0.725	0.036
3	0.800	0.901	1.062	0.211	0.745	0.044
4	1.018	1.068	1.152	0.153	0.778	0.068
5	0.896	0.974	1.101	0.184	0.762	0.054

Подмножество колонок задаётся теми же буквами. Например, только три вероятности (по построению их сумма равна 1) либо только латентное среднее как одномерный массив:

In [7]:

```
model.predict(X_show, kind='lmr').round(3)
```



```
Out [7]:
```

	prob_left	prob_interior	prob_right
0	0.443	0.548	0.009
1	0.067	0.778	0.155
2	0.240	0.725	0.036
3	0.211	0.745	0.044
4	0.153	0.778	0.068
5	0.184	0.762	0.054

```
In [8]: model.predict(X_show, kind='h') # single letter -> 1-D ndarray
```

```
Out [8]: array([0.14305362, 1.48881203, 0.70415823, 0.80034359, 1.01807358,
0.89647868])
```

Длинные имена — `kind='latent'`, `'censored'`, `'truncated'` — также допустимы и для обратной совместимости возвращают одномерный массив.

Предельные эффекты через `get_margeff`

Интерфейс предельных эффектов повторяет `get_margeff` из `statsmodels`: один метод с параметрами — где считать (`at`) и что возвращать (`method`). Итоговая таблица оформлена в привычном для `statsmodels` виде.

```
In [9]: # AME = average over the sample (at='overall'); statsmodels-style summary
print(model.get_margeff(at='overall', method='dydx', kind='censored').summary())
```

```

Censored Regression Marginal Effects
=====
Dep. Variable:          y
Method:             dydx
At:                overall
=====

```

	dy/dx	std err	z	P> z	[0.025	0.975]
x1	0.3600	0.012	30.227	0.000	0.337	0.383
x2	-0.1801	0.013	-14.113	0.000	-0.205	-0.155
x3	0.1426	0.013	10.805	0.000	0.117	0.168

```
=====
```

`at='overall'` соответствует AME, `at='mean'` — MEM. Ниже сравниваем цензурированный предельный эффект в разных точках оценки с латентным, который в точности равен β .

```
In [10]: import pandas as pd
summary = pd.DataFrame({
    'latent (=beta)': model.get_margeff(at='overall', kind='latent').margeff,
    'censored AME': model.get_margeff(at='overall', kind='censored').margeff,
    'censored MEM': model.get_margeff(at='mean', kind='censored').margeff,
    'truncated AME': model.get_margeff(at='overall', kind='truncated').margeff,
}, index=['x1', 'x2', 'x3'])
summary.round(4)
```

Out[10]:

	latent (=beta)	censored AME	censored MEM	truncated AME
x1	0.5126	0.3600	0.3973	0.2005
x2	-0.2565	-0.1801	-0.1988	-0.1003
x3	0.2030	0.1426	0.1573	0.0794

Параметр `method` управляет видом эффекта: `dydx` — производная, `eyex` — эластичность, `dyex` и `eydx` — полуэластичности. Для дискретных регрессоров доступен `dummy=True`: тогда возвращается разность $0 \rightarrow 1$, а не производная.

In [11]:

```
elas = model.get_margeff(at='overall', method='eyex', kind='censored')
elas.summary_frame().round(4)
```

Out[11]:

	eyex	std err	z	P> z	[0.025	0.975]
x1	-0.1238	0.0093	-13.3389	0.0	-0.1420	-0.1056
x2	-0.0317	0.0045	-7.0004	0.0	-0.0406	-0.0228
x3	-0.0193	0.0035	-5.4717	0.0	-0.0263	-0.0124

Предельные эффекты на вероятности зон

Помимо условного среднего можно вычислить, как регрессор сдвигает *вероятность* попадания в каждую зону: на левый порог ($P(Y = L) = \Phi(\alpha_L)$), внутрь интервала ($P(L < Y < R) = \Phi(\alpha_R) - \Phi(\alpha_L)$) или на правый ($P(Y = R) = 1 - \Phi(\alpha_R)$). Эти эффекты задаются ключом `kind` со значениями `'prob-left'`, `'prob-interior'`, `'prob-right'`. Поскольку три вероятности в сумме дают единицу, их предельные эффекты по построению складываются в ноль.

In [12]:

```
prob_effects = pd.DataFrame({
    'P(Y=L)': model.get_margeff(kind='prob-left').margeff,
    'P(L<Y<R)': model.get_margeff(kind='prob-interior').margeff,
    'P(Y=R)': model.get_margeff(kind='prob-right').margeff,
}, index=['x1', 'x2', 'x3'])
prob_effects['sum (=0)'] = prob_effects.sum(axis=1)
prob_effects.round(4)
```

Out[12]:

	P(Y=L)	P(L<Y<R)	P(Y=R)	sum (=0)
x1	-0.1221	0.0461	0.0760	0.0
x2	0.0611	-0.0231	-0.0380	0.0
x3	-0.0484	0.0183	0.0301	0.0

Положительный коэффициент перемещает наблюдения от левого порога к правому: эффект на `P(Y=L)` отрицателен, на `P(Y=R)` — положителен. Столбец

`sum (=0)` подтверждает, что ограничение «сумма равна нулю» выполняется численно.

Тест отношения правдоподобия

Проверим, нужен ли регрессор `x3` : подгоним ограниченную модель без него и сравним логарифмы правдоподобия.

```
In [13]: restricted = CensoredRegression(left=L, right=R).fit(X[:, :2], y)
res = lr_test(model, restricted)
print(res)
```

```
Likelihood-ratio test
LR statistic : 111.3544
df           : 1
p-value      : 4.949e-26
log-lik full : -3831.9928
log-lik rest.: -3887.6700
n            : 3000
```

Малое p-value отвергает ограничение, то есть переменную `x3` нельзя считать избыточной. В этом DGP истинный коэффициент при `x3` равен 0.2 и выборка велика, поэтому тест корректно отвергает гипотезу.

LR-тест с гипотезой в виде строки

Ручная подгонка ограниченной модели удобна лишь в простых случаях вроде «исключить одну колонку». Для совместных или составных ограничений `model.lr_test(hypotheses)` копирует интерфейс `f_test` из statsmodels и принимает линейные ограничения строкой — одиночные, совместные, а также арифметические выражения от имён параметров.

```
In [14]: # Drop a single regressor (same null as the two-model test above)
print(model.lr_test('x3 = 0'))
```

```
Likelihood-ratio test
LR statistic : 111.3544
df           : 1
p-value      : 4.949e-26
log-lik full : -3831.9928
log-lik rest.: -3887.6700
n            : 3000
```

```
In [15]: # Joint restriction: x2 AND x3 are zero (composite null)
print(model.lr_test('(x2 = 0), (x3 = 0)'))
```

```
Likelihood-ratio test
LR statistic : 286.4290
df           : 2
p-value      : 6.35e-63
log-lik full : -3831.9928
log-lik rest.: -3975.2073
n            : 3000
```

```
In [16]: # Composite restriction with arithmetic: x1 - 2*x2 = 0
# (equivalent to testing whether x1 equals twice x2)
print(model.lr_test('x1 - 2*x2 = 0'))
```

```
Likelihood-ratio test
LR statistic : 549.6200
df           : 1
p-value      : 1.523e-121
log-lik full : -3831.9928
log-lik rest.: -4106.8028
n            : 3000
```

При каждом вызове задача оптимизации решается заново с линейным ограничением $R\hat{\theta} = r$ (через `scipy.optimize` и `LinearConstraint`), а в качестве результата сообщается асимптотическая LR-статистика $2(\ell_{\text{full}} - \ell_{\text{rest.}}) \sim \chi_q^2$.

3.2.2 Классический Tobit на данных Fair (1978)

```
In [17]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from censrunc import CensoredRegression

data = sm.datasets.fair.load_pandas().data
print('n =', len(data))
print(f"share with affairs == 0: {(data['affairs'] == 0).mean():.1%}")
data.head()
```

```
n = 6366
share with affairs == 0: 67.8%
```

```
Out[17]:
```

	rate_marriage	age	ysr_married	children	religious	educ	occupation	occupation
0	3.0	32.0	9.0	3.0	3.0	17.0	2.0	
1	3.0	27.0	13.0	3.0	1.0	14.0	3.0	
2	4.0	22.0	2.5	0.0	1.0	16.0	3.0	
3	4.0	37.0	16.5	4.0	3.0	16.0	5.0	
4	5.0	27.0	9.0	1.0	1.0	14.0	3.0	

Спецификация

В духе Fair (1978) рассмотрим регрессию числа случаев супружеской измены на стандартный набор демографических и семейных признаков.

```
In [18]: covariates = ['rate_marriage', 'age', 'ysr_married', 'children',
                    'religious', 'educ', 'occupation', 'occupation_husb']
X = data[covariates]
y = data['affairs']
```

Оценивание

Левый порог `CensoredRegression` равен нулю; правый порог не задаётся.

```
In [19]: model = CensoredRegression(left=0.0).fit(X, y)
print(model.summary())
```

```
=====
Censored Regression Results
=====
Dep. Variable:          y          No. Observations:      6366
Model:                  CensoredRegression      Df Model:          9
Method:                 MLE          Df Residuals:       6356
Date:                  Wed, 03 Jun 2026      Log-Likelihood:    -7804.3803
Time:                  02:06:06          LL-Null:          -8155.0240
AIC:                   15628.7605        LLR p-value:       3.780e-146
BIC:                   15696.3478        Pseudo R-squ.:    0.0430
Left threshold:        0                Right threshold:   none
=====
```

	coef	std err	z	P> z	[0.025	0.975]
sigma	4.4990	0.0771	58.3463	0.0000	4.3478	4.6501
const	7.8354	0.7135	10.9818	0.0000	6.4370	9.2338
rate_marriage	-1.5313	0.0734	-20.8514	0.0000	-1.6752	-1.3873
age	-0.1051	0.0248	-4.2393	0.0000	-0.1537	-0.0565
yrs_married	0.1282	0.0264	4.8576	0.0000	0.0765	0.1799
children	-0.0276	0.0771	-0.3576	0.7206	-0.1788	0.1236
religious	-0.9434	0.0849	-11.1077	0.0000	-1.1099	-0.7769
educ	-0.0857	0.0376	-2.2772	0.0228	-0.1594	-0.0119
occupation	0.3125	0.0819	3.8150	0.0001	0.1520	0.4731
occupation_husb	0.0143	0.0555	0.2578	0.7966	-0.0944	0.1230

```
=====
Left-censored observations:      4313 (67.75%)
Right-censored observations:      0 ( 0.00%)
Uncensored observations:        2053 (32.25%)
Optimiser converged:            True
=====
```

Предельные эффекты

Поскольку отклик цензурирован слева на нуле, цензурированные предельные эффекты по модулю меньше латентных. Латентные (нецензурированные) коэффициенты описывают шкалу склонности, а цензурированные предельные эффекты — *ожидаемое число* измен.

```
In [20]: ame_cens = model.ame(kind='censored').to_dataframe()
ame_lat = model.ame(kind='latent').to_dataframe()
pd.concat([
    ame_lat[['dy/dx']].rename(columns={'dy/dx': 'latent'}),
    ame_cens[['dy/dx', 'std err', 'P>|z|']].rename(columns={'dy/dx': 'censored E[Y]'}),
], axis=1).round(4)
```

Out [20]:

	latent	censored E[Y]	std err	P> z
rate_marriage	-1.5313	-0.4326	0.0213	0.0000
age	-0.1051	-0.0297	0.0070	0.0000
yrs_married	0.1282	0.0362	0.0074	0.0000
children	-0.0276	-0.0078	0.0218	0.7206
religious	-0.9434	-0.2665	0.0243	0.0000
educ	-0.0857	-0.0242	0.0106	0.0228
occupation	0.3125	0.0883	0.0232	0.0001
occupation_husb	0.0143	0.0040	0.0157	0.7966

Проверка с OLS

На цензурированной выборке OLS занижает оценки наклонов (Greene, 1981); Tobit MLE это смещение устраняет.

In [21]:

```
X_int = sm.add_constant(X)
ols = sm.OLS(y, X_int).fit()
pd.DataFrame({
    'OLS coef':    ols.params.values,
    'Tobit coef':  model.coef_,
}, index=ols.params.index).round(4)
```

Out [21]:

	OLS coef	Tobit coef
const	3.6235	7.8354
rate_marriage	-0.4205	-1.5313
age	-0.0146	-0.1051
yrs_married	-0.0160	0.1282
children	-0.0171	-0.0276
religious	-0.2437	-0.9434
educ	-0.0174	-0.0857
occupation	0.0658	0.3125
occupation_husb	0.0040	0.0143

3.2.3 Усечённая регрессия

In [22]:

```
import numpy as np
import pandas as pd
from censtrunc import TruncatedRegression
```

```
rng = np.random.default_rng(7)
n_population = 8000
```

Моделирование усечённой выборки

Сгенерируем большую генеральную совокупность и оставим только наблюдения с $0 < Y^* < 2.5$. В этом DGP двустороннее усечение «переживают» около 60 % исходных наблюдений.

```
In [23]: X_pop = rng.normal(size=(n_population, 2))
beta_true = np.array([1.0, 0.5, -0.3])
y_star = beta_true[0] + X_pop @ beta_true[1:] + rng.normal(scale=1.0, size=n_population)
L, R = 0.0, 2.5
mask = (y_star > L) & (y_star < R)
X = X_pop[mask]
y = y_star[mask]
print(f'Observed after truncation: {len(y)} / {n_population}')
```

Observed after truncation: 5741 / 8000

Подгонка усечённой регрессии

```
In [24]: model = TruncatedRegression(left=L, right=R).fit(
        X, y, feature_names=['x1', 'x2']
    )
print(model.summary())
```

```
=====
                        Truncated Regression Results
=====
Dep. Variable:          y                No. Observations:   5741
Model:                TruncatedRegression    Df Model:         3
Method:               MLE                  Df Residuals:    5737
Date:                 Wed, 03 Jun 2026      Log-Likelihood:   -4830.9612
Time:                 02:06:06              LL-Null:         -5136.1192
AIC:                  9669.9225             LLR p-value:      2.962e-133
BIC:                  9696.5440             Pseudo R-squ.:   0.0594
Left truncation:      0                    Right truncation: 2.5
=====
```

	coef	std err	z	P> z	[0.025	0.975]
sigma	0.9944	0.0316	31.5070	0.0000	0.9326	1.0563
const	0.9946	0.0245	40.6756	0.0000	0.9467	1.0425
x1	0.4881	0.0335	14.5918	0.0000	0.4226	0.5537
x2	-0.3105	0.0268	-11.6040	0.0000	-0.3629	-0.2580

```
=====
Optimiser converged:      True
=====
```

Сравнение: OLS на усечённой выборке смещён

OLS на усечённой выборке трактует её как обычную выборку из неограниченной генеральной совокупности — в итоге оценки наклонов смещены и, как правило, сжаты к нулю.

```
In [25]: X_int = np.column_stack([np.ones(len(y)), X])
ols_beta, *_ = np.linalg.lstsq(X_int, y, rcond=None)

comparison = pd.DataFrame({
    'true':      beta_true,
```

```

    'OLS (biased)': ols_beta,
    'Truncated MLE': model.coef_,
}, index=['const', 'x1', 'x2'])
comparison['OLS bias'] = comparison['OLS (biased)'] - comparison['true']
comparison['Truncated MLE bias'] = comparison['Truncated MLE'] - comparison['true']
comparison.round(4)

```

Out [25]:

	true	OLS (biased)	Truncated MLE	OLS bias	Truncated MLE bias
const	1.0	1.1475	0.9946	0.1475	-0.0054
x1	0.5	0.1952	0.4881	-0.3048	-0.0119
x2	-0.3	-0.1241	-0.3105	0.1759	-0.0105

Прогнозы

У усечённой модели содержательный смысл имеют только две величины (точечных масс на порогах нет — столбцы вероятностей зон неприменимы):

- **h** (latent): $X'\hat{\beta}$ — среднее в генеральной совокупности
- **t** (truncated): $\mathbb{E}[Y|X, L < Y < R]$

In [26]:

```

preds = model.predict(X[:6]) # default = 'ht' -> both columns
preds.insert(0, 'observed', y[:6])
preds.round(3)

```

Out [26]:

	observed	latent	truncated
0	1.590	0.902	1.104
1	0.317	1.137	1.202
2	1.514	1.081	1.178
3	1.376	0.947	1.122
4	1.085	0.764	1.047
5	0.074	0.481	0.936

3.2.4 Иллюстрация: усечение и цензура

In [27]:

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from censtrunc import CensoredRegression, TruncatedRegression
from censtrunc._means import value_for_kind
from censtrunc._utils import _prepare_design_matrix

rng = np.random.default_rng(2026)
n = 250
x = rng.uniform(-10, 10, size=n)
sigma_true = 2.0

```



```
y_star = 1.0 * x + rng.normal(scale=sigma_true, size=n)
L, R = -5.0, 5.0
```

Усечённая выборка: наблюдения с $y \notin (L, R)$ полностью исключены.

Цензурированная выборка: те же наблюдения заменены ближайшим порогом.

Латентная линейная зависимость $y = x$ в обоих случаях одна и та же.

```
In [28]: mask = (y_star > L) & (y_star < R)
x_t, y_t = x[mask], y_star[mask]
x_c, y_c = x.copy(), np.clip(y_star, L, R)

mt = TruncatedRegression(left=L, right=R).fit(x_t.reshape(-1, 1), y_t)
mc = CensoredRegression(left=L, right=R).fit(x_c.reshape(-1, 1), y_c)
print(f'truncated: n={len(y_t)}, beta_hat = {mt.coef_.round(3)}')
print(f'censored: n={len(y_c)}, beta_hat = {mc.coef_.round(3)}')
```

```
truncated: n=146, beta_hat = [-0.003  0.946]
censored: n=250, beta_hat = [0.059  0.939]
```

Построим доверительную полосу прогнозов по асимптотической неопределённости. Возьмём 200 векторов параметров из асимптотического нормального распределения $\hat{\theta}$, для каждого посчитаем кривую условного среднего (усечённого слева, цензурированного справа) и нарисуем кривые полупрозрачными. Скопления на порогах справа выделены красным.

```
In [29]: def prediction_band(model, x_grid, kind, n_draws=200, seed=0):
    rng = np.random.default_rng(seed)
    samples = rng.multivariate_normal(model.params_, model.cov_params_, size=n_draws)
    samples = samples[samples[:, 0] > 0] # keep only feasible sigma > 0
    Xg, _ = _prepare_design_matrix(x_grid.reshape(-1, 1), fit_intercept=True)
    curves = np.empty((samples.shape[0], x_grid.size))
    for i, theta in enumerate(samples):
        sigma, beta = theta[0], theta[1:]
        curves[i] = value_for_kind(
            beta, sigma, Xg,
            model._left, model._right, model._has_left, model._has_right, kind,
        )
    return curves

x_grid = np.linspace(-10, 10, 300)
curves_t = prediction_band(mt, x_grid, kind='truncated')
curves_c = prediction_band(mc, x_grid, kind='censored')
```

```
In [30]: fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), sharey=True)
for ax in axes:
    ax.set_facecolor('#eaeaf2')
    ax.grid(True, color='white', linewidth=1)
    ax.axhline(L, color='crimson', linestyle='--', linewidth=1.2)
    ax.axhline(R, color='crimson', linestyle='--', linewidth=1.2)
    ax.set_xlim(-10, 10); ax.set_ylim(-10, 10)
    ax.set_xlabel('x'); ax.set_ylabel('y')
    ax.plot(x_grid, x_grid, color='black', linewidth=2.5, label='True')

# Left panel: truncated
ax = axes[0]
ax.set_title('Truncated data')
for c in curves_t:
    ax.plot(x_grid, c, color='steelblue', alpha=0.04, linewidth=1)
ax.scatter(x_t, y_t, c='black', s=14, zorder=5)
ax.legend(loc='upper left')

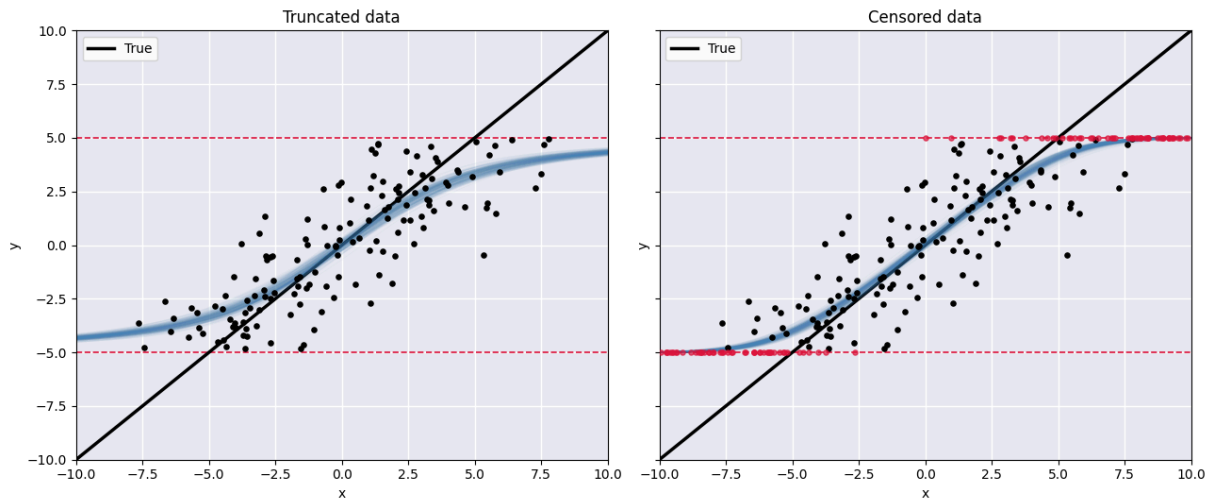
# Right panel: censored
ax = axes[1]
ax.set_title('Censored data')
```

```

for c in curves_c:
    ax.plot(x_grid, c, color='steelblue', alpha=0.04, linewidth=1)
is_at_bound = (y_c == L) | (y_c == R)
ax.scatter(x_c[~is_at_bound], y_c[~is_at_bound], c='black', s=14, zorder=5)
ax.scatter(x_c[is_at_bound], y_c[is_at_bound], c='crimson', s=14, zorder=5, alpha=0.7)
ax.legend(loc='upper left')

plt.tight_layout(); plt.show()

```



Что обратить внимание.

- Чёрная линия — истинная зависимость $y^* = x$.
- Синяя полоса — прогноз MLE с асимптотической неопределённостью. **Слева** она загибается внутрь интервала (L, R) , потому что усечённое среднее $\mathbb{E}[Y \mid X, L < Y < R]$ смещается к центру интервала; **справа** становится более пологой у порогов, поскольку цензурированное среднее $\mathbb{E}[Y \mid X]$ совмещает латентную линейную часть с точечными массами.
- Красные точки на правой панели — скопления цензурированных наблюдений на $y = L$ и $y = R$. В усечённой выборке таких скоплений нет, поскольку эти наблюдения отсутствуют как таковые.
- Обе оценочные кривые близки к истинной во внутренней области, где данных больше всего; у обеих падает точность вблизи и за пороговыми, но ни одна не испытывает столь сильной аттенуации к нулю, как OLS.

3.3 Эмпирическая проверка корректности

Три независимые проверки цензурированной MLE:

1. **Предел без цензуры** — если пороги отодвинуть за пределы диапазона данных, ни одно наблюдение не попадает под цензуру, и MLE обязана совпасть с OLS.
2. **Probit-предел** — при $L \approx R$ цензурированный логарифм правдоподобия сводится к probit-правдоподобию для индикатора $\mathbf{1}\{Y^* > c\}$ с

$\beta_{\text{tobit}}/\sigma_{\text{tobit}} = \beta_{\text{probit}}$ (Hansen, 2022, §27.4).

3. **Кросс-валидация против R** — `survival::survreg` (движок, на котором построен `AER::tobit`) и `truncreg::truncreg` решают ту же задачу MLE; две корректные реализации обязаны сойтись в одну и ту же точку с точностью оптимизатора.

```
In [31]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import CensoredRegression, TruncatedRegression

rng = np.random.default_rng(0)
n = 1000
X = rng.normal(size=(n, 3))
y = 2.0 + 1.5*X[:,0] - 0.7*X[:,1] + 0.3*X[:,2] + rng.normal(scale=1.3, size=n)
```

Без цензуры $\Rightarrow$ OLS

Возьмём `left=-1e6`, `right=1e6` — ни одно наблюдение не попадёт под цензуру. Цензурированный логарифм правдоподобия в этом пределе сводится к гауссовому, а его максимум совпадает с OLS-оценкой.

```
In [32]: ols = sm.OLS(y, sm.add_constant(X)).fit()
cens = CensoredRegression(left=-1e6, right=1e6).fit(X, y)
trunc = TruncatedRegression(left=-1e6, right=1e6).fit(X, y)

pd.DataFrame({
    'OLS': np.asarray(ols.params),
    'Censored MLE': cens.coef_,
    'Truncated MLE': trunc.coef_,
}, index=['const', 'x1', 'x2', 'x3']).round(6)
```

```
Out [32]:
```

	OLS	Censored MLE	Truncated MLE
const	2.046512	2.046482	2.046512
x1	1.434191	1.434221	1.434191
x2	-0.626598	-0.626603	-0.626598
x3	0.345980	0.345982	0.345980

Колонки совпадают до нескольких знаков. Дополнительно проверим максимальное расхождение и совпадение параметра масштаба и логарифма правдоподобия, используя MLE-масштаб $\hat{\sigma} = \sqrt{\text{SSR}/n}$.

```
In [33]: ols_beta = np.asarray(ols.params)
print(f'max |beta_censored - beta_OLS| = {np.max(np.abs(cens.coef_ - ols_beta)):.2e}')
print(f'max |beta_truncated - beta_OLS| = {np.max(np.abs(trunc.coef_ - ols_beta)):.2e}')
print(f'|sigma_censored - sqrt(SSR/n)| = {abs(cens.sigma_ - np.sqrt(ols.ssr/n)):.2e}')
print(f'|loglik_censored - loglik_OLS| = {abs(cens.llf_ - ols.llf):.2e}')

max |beta_censored - beta_OLS| = 3.00e-05
max |beta_truncated - beta_OLS| = 4.44e-16
|sigma_censored - sqrt(SSR/n)| = 7.55e-07
|loglik_censored - loglik_OLS| = 5.90e-07
```

Все расхождения — на уровне точности оптимизатора. В отсутствие цензуры оценщик сводится к OLS, как и должно быть; следовательно, логарифм правдоподобия и его оптимизация реализованы корректно.

С цензурой: согласие с R и расхождение с OLS

Решающая проверка — действительно цензурированные данные. На каждый коэффициент сравним четыре числа:

- **true** — истинное значение из DGP;
- **censtrunc** — оценка MLE из нашего пакета;
- **R survreg** — гауссова реализация в `survreg` (на ней построен `AER::tobit`), посчитанная полностью независимым кодом;
- **OLS** — наивный МНК, смещённый при цензуре.

Две корректные реализации MLE обязаны сойтись к *одному и тому же* единственному максимуму, так что `censtrunc` и R должны совпасть с точностью оптимизатора, а OLS — отличаться от обеих и быть смещённым.

```
In [34]: import shutil, subprocess, json, tempfile, os
         from pathlib import Path

         rng2 = np.random.default_rng(20260527)
         nc = 4000
         Xc = rng2.normal(size=(nc, 2))
         y_star_c = 1.0 + 0.7*Xc[:, 0] - 0.4*Xc[:, 1] + rng2.normal(size=nc)
         Lc, Rc = 0.0, 2.5
         yc = np.clip(y_star_c, Lc, Rc)

         cens = CensoredRegression(left=Lc, right=Rc).fit(Xc, yc)
         ols_c = np.asarray(sm.OLS(yc, sm.add_constant(Xc)).fit().params)

         # Run R's survreg via the bundled reference script, if R is available.
         def _find_r_script():
             for c in [Path('tests/reference/fit_tobit.R'),
                       Path('..tests/reference/fit_tobit.R')]:
                 if c.exists():
                     return c
             return None

         r_beta = r_sigma = None
         rscript, script_path = shutil.which('Rscript'), _find_r_script()
         if rscript and script_path:
             fd, csvp = tempfile.mkstemp(suffix='.csv'); os.close(fd)
             np.savetxt(csvp, np.column_stack([yc, Xc]), delimiter=',', header='y,x1,x2', comments='')
             try:
                 out = subprocess.run([rscript, str(script_path), csvp, str(Lc), str(Rc)],
                                     capture_output=True, text=True, timeout=120, check=True)
                 rt = json.loads(out.stdout)['tobit']
                 r_beta = np.array([rt['coef']['(Intercept)'], rt['coef']['x1'], rt['coef']['x2']])
                 r_sigma = float(rt['scale'])
             except Exception:
                 r_beta = None
             finally:
                 os.remove(csvp)

In [35]: if r_beta is not None:
         table = pd.DataFrame({
```

```

    'true':          [1.0, 0.7, -0.4],
    'censtrunc':     cens.coef_,
    'R survreg':     r_beta,
    'OLS (biased)':  ols_c,
}, index=['const', 'x1', 'x2'])
display(table.round(6))
print(f'max |censtrunc - R| = {np.max(np.abs(cens.coef_ - r_beta)).2e} (independent solvers, same optimum)')
print(f'|sigma_censtrunc - sigma_R| = {abs(cens.sigma_ - r_sigma).2e}')
print(f'max |censtrunc - OLS| = {np.max(np.abs(cens.coef_ - ols_c)).3f} (Tobit correction is real)')
else:
    print('R not available at build time.')
    print('The live comparison runs in the test suite: tests/test_r_reference.py')
    print('censtrunc estimates:', cens.coef_.round(4))
    print('OLS (biased):', ols_c.round(4))

```

	true	censtrunc	R survreg	OLS (biased)
const	1.0	1.014284	1.014284	1.090364
x1	0.7	0.701331	0.701331	0.470177
x2	-0.4	-0.410105	-0.410105	-0.277838

```

max |censtrunc - R| = 3.05e-08 (independent solvers, same optimum)
|sigma_censtrunc - sigma_R| = 2.41e-08
max |censtrunc - OLS| = 0.231 (Tobit correction is real)

```

Колонки `censtrunc` и `R` совпадают примерно до шести знаков, и при этом `max |censtrunc - R|` — крошечное ненулевое число порядка 10^{-8} : два независимых оптимизатора пришли в один и тот же максимум, оставаясь не побитово идентичными. Обе процедуры восстанавливают истинные коэффициенты, тогда как OLS заметно смещён к нулю — это и есть аттенуация при цензуре, описанная Greene (1981).

Предел probit: при узких порогах Tobit сводится к probit

Возьмём цензурированную регрессию с $L = R = c$. Когда внутренней зоны нет, логарифм правдоподобия приводится к виду

$$\ell(\beta, \sigma) = \sum_i \mathbf{1}\{y_i = c\} \log \Phi\left(\frac{c - X_i' \beta}{\sigma}\right) + \mathbf{1}\{y_i > c\} \log \left[1 - \Phi\left(\frac{c - X_i' \beta}{\sigma}\right)\right],$$

а это в точности логарифм правдоподобия probit для индикатора $D_i = \mathbf{1}\{Y_i^* > c\}$, причём идентифицируется $\beta_{\text{probit}} = \beta_{\text{tobit}} / \sigma_{\text{tobit}}$ (Hansen, 2022, §27.4).

Положить $L = R$ буквально невозможно (тогда σ не идентифицируется), однако достаточно узкой полосы $[c - \varepsilon, c + \varepsilon]$. Немногочисленные наблюдения внутри полосы идентифицируют σ ; остальные ведут себя как бинарный исход. При $\varepsilon \rightarrow 0$ должно выполняться $\hat{\beta}_{\text{tobit}} / \hat{\sigma}_{\text{tobit}} \rightarrow \hat{\beta}_{\text{probit}}$.

```

In [36]: from statsmodels.discrete.discrete_model import Probit

rng3 = np.random.default_rng(20250601)

```

```

n3 = 8000
Xp = rng3.normal(size=(n3, 2))
beta_p = np.array([0.4, 1.2, -0.8])
y_star_p = beta_p[0] + Xp @ beta_p[1:] + rng3.normal(size=n3)
y_bin = (y_star_p > 0.0).astype(float)
probit = Probit(y_bin, sm.add_constant(Xp)).fit(disp=False)

def fit_band(eps: float):
    y_obs = np.clip(y_star_p, -eps, eps)
    m = CensoredRegression(left=-eps, right=eps).fit(Xp, y_obs)
    return m.coef_ / m.sigma_ # the ratio that should match probit

table = pd.DataFrame({
    'probit beta': np.asarray(probit.params),
    'tobit / sigma (eps=0.5)': fit_band(0.50),
    'tobit / sigma (eps=0.1)': fit_band(0.10),
    'tobit / sigma (eps=0.05)': fit_band(0.05),
}, index=['const', 'x1', 'x2'])
table.round(4)

```

```

Out[36]:

```

	probit beta	tobit / sigma (eps=0.5)	tobit / sigma (eps=0.1)	tobit / sigma (eps=0.05)
const	0.3791	0.3755	0.3728	0.3744
x1	1.2218	1.1923	1.2134	1.2192
x2	-0.7774	-0.7726	-0.7754	-0.7783

По мере сужения полосы отношение «tobit/sigma» приближается к probit-оценке. Свободный член отклоняется сильнее, чем наклоны — это эффект конечной полосы порядка ε/σ ; уже при $\varepsilon = 0.05$ наклоны совпадают в пределах нескольких процентов. Тем самым отдельно проверяются граничные члены ℓ — помимо проверки «без цензуры», сделанной выше.

Часть Б. Модель селективного отбора Хекмана

4.1 Теория

Задача с отбором

Пусть зависимая переменная Y наблюдается только тогда, когда индивид *отбирается* в выборку по ненаблюдаемому правилу. Канонический пример — уравнение зарплаты: Y = логарифм зарплаты — наблюдается лишь у занятых. Модель имеет вид

$$Y^* = X'\beta + e, \quad S^* = Z'\gamma + u, \quad S = \mathbf{1}\{S^* > 0\},$$

$$Y = Y^* \text{ при } S = 1, \text{ иначе не наблюдается, } (e, u) \sim \mathcal{N}\left(0, \begin{pmatrix} \sigma^2 & \rho\sigma \\ \rho\sigma & 1 \end{pmatrix}\right).$$

Дисперсия u нормирована на единицу для идентифицируемости (стандартное probit-соглашение). При $\rho \neq 0$ OLS на отобранной подвыборке смещён, поскольку

$$\mathbb{E}[Y \mid X, Z, S = 1] = X'\beta + \rho\sigma \lambda(Z'\gamma),$$

где $\lambda(t) = \phi(t)/\Phi(t)$ — обратный коэффициент Миллса. Если опустить $\lambda(Z'\gamma)$ в регрессии (как делает наивный OLS), получается классическое смещение из-за пропущенной переменной.

Двухшаговая процедура Хекмана

1. **Probit-регрессия** S по Z даёт оценку $\hat{\gamma}$.
2. **Обратный коэффициент Миллса** $\hat{\lambda}_i = \phi(Z'_i\hat{\gamma})/\Phi(Z'_i\hat{\gamma})$ вычисляется для отобранных наблюдений.
3. **OLS-регрессия** y_i по $(X_i, \hat{\lambda}_i)$ на отобранной подвыборке даёт оценки $\hat{\beta}$ и $\hat{\rho}\hat{\sigma}$.

Наивные стандартные ошибки второго шага занижены: они игнорируют дисперсию $\hat{\gamma}$ из probit-шага. Парный бутстрап (`model.bootstrap`) восполняет этот пробел.

Совместная MLE

Полный логарифм правдоподобия (Hansen, 2022, §27.10):

$$\ell = \sum_{S_i=0} \log[1 - \Phi(Z'_i\gamma)] + \sum_{S_i=1} \left\{ \log \Phi\left(\frac{Z'_i\gamma + (\rho/\sigma)(Y_i - X'_i\beta)}{\sqrt{1 - \rho^2}}\right) - \frac{1}{2}\log(2\pi\sigma^2) - \frac{1}{2}\left(\frac{Z'_i\gamma + (\rho/\sigma)(Y_i - X'_i\beta)}{\sqrt{1 - \rho^2}}\right)^2 \right\}$$

Метод асимптотически эффективен, но несколько медленнее двухшагового.

Внутренняя оптимизация использует $\rho = \tanh(\eta)$, чтобы условие $|\rho| < 1$ выполнялось автоматически.

Шесть прогнозных величин

`HeckitRegression.predict()` отдаёт шесть величин через однобуквенный код `kind`; по умолчанию `'snpohu'` возвращает все шесть в DataFrame.

Уравнение отбора (использует Z):

- **s** — вероятность включения $\Phi(Z'\hat{\gamma})$
- **n** — вероятность не включения $1 - \Phi(Z'\hat{\gamma})$
- **p** — латентная склонность к включению $Z'\hat{\gamma}$

Уравнение исхода (использует X , иногда Z):

- **o** — наблюдаемое условное среднее $\mathbb{E}[Y \mid X, Z, S = 1] = X'\hat{\beta} + \hat{\rho}\hat{\sigma}\lambda(Z'\hat{\gamma})$
- **h** — латентное безусловное $\mathbb{E}[Y^* \mid X] = X'\hat{\beta}$
- **u** — условное среднее у неотобранных
 $\mathbb{E}[Y^* \mid X, Z, S = 0] = X'\hat{\beta} - \hat{\rho}\hat{\sigma}\phi(Z'\hat{\gamma})/[1 - \Phi(Z'\hat{\gamma})]$

Предельные эффекты

Четыре стандартных типа из Greene, *Econometric Analysis*, §19.5, доступны через `model.get_margeff(kind=...)`:

kind	Дифференцирует
'latent'	$E[Y^* \mid X] = X'\beta$ — только X-сторона
'conditional'	$E[Y \mid X, Z, S = 1]$
'unconditional'	$E[Y \cdot S \mid X, Z]$ $= \Phi(Z'\gamma)X'\beta$ $+ \rho\sigma\phi(Z'\gamma)$
'prob-selected'	$P(S = 1 \mid Z)$ $= \Phi(Z'\gamma)$ — только Z-сторона

При $\delta(t) = \lambda(t)(t + \lambda(t))$ (так что $d\lambda/dt = -\delta$) построчная производная условного среднего по переменной v имеет вид

$$\frac{\partial E[Y \mid S = 1]}{\partial v} = \beta_v \mathbf{1}\{v \in X\} - \gamma_v \rho \sigma \delta(Z'\gamma) \mathbf{1}\{v \in Z\}.$$

Переменные, входящие в оба уравнения (сопоставляются по имени), складывают свои X- и Z-производные. Стандартные ошибки получаются дельта-методом по совместной ковариации MLE.

4.2 Применение

Демонстрация на симулированных данных, а затем на каноническом наборе Mroz (1987) для уравнения зарплаты: как вызывать двухшаговый и MLE-оценщики, как запросить шесть прогнозных величин, как пользоваться формулами в стиле `patsy`, как получить корректные стандартные ошибки двухшаговой оценки бутстрапом и как читать таблицу предельных эффектов.

```
In [37]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import HeckitRegression

rng = np.random.default_rng(42)
```



```

n = 4000
# Shared regressor (in both equations); two exclusive regressors
shared = rng.normal(size=n)
x_only = rng.normal(size=n)      # in the outcome eq. only
z_only = rng.normal(size=n)      # in the selection eq. only (exclusion)

beta_true = np.array([1.0, 0.5, -0.3]) # const, shared, x_only
gamma_true = np.array([0.0, 0.3, 0.6]) # const, shared, z_only
rho_true, sigma_true = 0.6, 1.0

Sigma = np.array([[sigma_true**2, rho_true*sigma_true],
                  [rho_true*sigma_true, 1.0]])
errs = rng.multivariate_normal([0.0, 0.0], Sigma, size=n)
e, u = errs[:, 0], errs[:, 1]

X = np.column_stack([shared, x_only])
Z = np.column_stack([shared, z_only])
S = (gamma_true[0] + Z @ gamma_true[1:] + u > 0).astype(int)
y_full = beta_true[0] + X @ beta_true[1:] + e
y = np.where(S == 1, y_full, np.nan)
print(f'Selected (S=1): {S.sum()} / {n} ({S.mean():.1%})')

```

Selected (S=1): 2022 / 4000 (50.5%)

Почему OLS на отобранной подвыборке смещён

При $\rho > 0$ ошибки u и e коррелированы, поэтому наблюдения с высоким e в среднем имеют высокий u и потому чаще попадают в выборку. Условное среднее на отобранных принимает вид

$$\mathbb{E}[Y \mid X, S = 1] = X'\beta + \rho\sigma\lambda(Z'\gamma),$$

где $\lambda(\cdot)$ — обратный коэффициент Миллса. Если опустить $\lambda(Z'\gamma)$ в регрессии (как делает наивный OLS), получается классическое смещение из-за пропущенной переменной.

```

In [38]: sel = ~np.isnan(y)
X_full = sm.add_constant(X)
ols = sm.OLS(y[sel], X_full[sel]).fit()
print('OLS on selected subsample (biased):')
print(np.asarray(ols.params).round(4))
print('truth:', beta_true)

```

```

OLS on selected subsample (biased):
[ 1.3651  0.4366 -0.3037]
truth: [ 1.   0.5 -0.3]

```

Двухшаговая процедура Хекмана

Алгоритм состоит из трёх шагов:

1. Probit-регрессия S по Z даёт оценку $\hat{\gamma}$.
2. Для отобранных наблюдений вычисляется обратный коэффициент Миллса $\hat{\lambda}_i = \phi(Z_i'\hat{\gamma})/\Phi(Z_i'\hat{\gamma})$.
3. OLS-регрессия y_i по $(X_i, \hat{\lambda}_i)$ на отобранной подвыборке даёт оценки $\hat{\beta}$ и $\hat{\rho}\hat{\sigma}$.

```

In [39]: m_two = HeckitRegression(method='twostep').fit(y, X, Z)

```

```
print(m_two.summary())
```

```

Heckman Selection Regression
=====
Method:                twostep                No. Observations:    4000
Selected (S=1):        2022                  Censored (S=0):      1978
sigma_e:               0.952857              rho:                0.531381
=====
Outcome equation (Y* = X * beta + e)
-----
              coef      std err          z      P>|z|      [0.025    0.975]
-----
const         1.0267      0.0491     20.9278    0.0000      0.9306    1.1229
x1             0.4903      0.0223     21.9471    0.0000      0.4465    0.5341
x2            -0.3010      0.0209    -14.4109    0.0000     -0.3419   -0.2601
-----
Selection equation (S* = Z * gamma + u, var(u) = 1)
-----
              coef      std err          z      P>|z|      [0.025    0.975]
-----
const         0.0333      0.0213      1.5611    0.1185     -0.0085    0.0751
x1            0.2585      0.0218     11.8357    0.0000      0.2157    0.3013
x2            0.6260      0.0242     25.8990    0.0000      0.5786    0.6734
=====
Note: two-step second-stage standard errors are naive; call .bootstrap(y, X, Z) for proper Heckman-correc
=====

```

Совместный метод максимального правдоподобия

Максимизируется совместный логарифм правдоподобия (Hansen, 2022, §27.10):

$$\ell = \sum_{S_i=0} \log[1 - \Phi(Z_i'\gamma)] + \sum_{S_i=1} \left\{ \log \Phi \left(\frac{Z_i'\gamma + (\rho/\sigma)(Y_i - X_i'\beta)}{\sqrt{1 - \rho^2}} \right) - \frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2} \right\}$$

Метод асимптотически эффективен, но несколько медленнее двухшагового.

```
In [40]: m_ml = HeckitRegression(method='mle').fit(y, X, Z)
print(m_ml.summary())
```

```

Heckman Selection Regression
=====
Method:                mle                No. Observations:    4000
Selected (S=1):        2022                  Censored (S=0):      1978
sigma_e:               0.958607              rho:                0.552343
Log-Likelihood:        -4924.8374
=====
Outcome equation (Y* = X * beta + e)
-----
              coef      std err          z      P>|z|      [0.025    0.975]
-----
const         1.0108      0.0446     22.6544    0.0000      0.9234    1.0983
x1             0.4925      0.0214     23.0340    0.0000      0.4506    0.5344
x2            -0.3017      0.0192    -15.7382    0.0000     -0.3393   -0.2642
-----
Selection equation (S* = Z * gamma + u, var(u) = 1)
-----
              coef      std err          z      P>|z|      [0.025    0.975]
-----
const         0.0345      0.0213      1.6183    0.1056     -0.0073    0.0763
x1            0.2598      0.0218     11.9411    0.0000      0.2171    0.3024
x2            0.6242      0.0241     25.8950    0.0000      0.5769    0.6714
=====

```

Сводное сравнение оценок

Сопоставим три оценки на одних и тех же данных. Двухшаговый Хекит и MLE восстанавливают β_{shared} заметно ближе к истинному значению, чем OLS, и попутно дают оценки ρ и σ .

```
In [41]: comparison = pd.DataFrame({
    'true': beta_true,
    'OLS (biased)': np.asarray(ols.params),
    'Heckit 2step': m_two.coef_,
    'Heckit MLE': m_ml.coef_,
    }, index=['const', 'shared', 'x_only'])
comparison['OLS error'] = comparison['OLS (biased)'] - comparison['true']
comparison['2step error'] = comparison['Heckit 2step'] - comparison['true']
comparison['MLE error'] = comparison['Heckit MLE'] - comparison['true']
comparison.round(4)
```

```
Out[41]:
```

	true	OLS (biased)	Heckit 2step	Heckit MLE	OLS error	2step error	MLE error
const	1.0	1.3651	1.0267	1.0108	0.3651	0.0267	0.0108
shared	0.5	0.4366	0.4903	0.4925	-0.0634	-0.0097	-0.0075
x_only	-0.3	-0.3037	-0.3010	-0.3017	-0.0037	-0.0010	-0.0017

```
In [42]: pd.DataFrame({
    'true': [rho_true, sigma_true],
    'two-step': [m_two.rho_, m_two.sigma_],
    'MLE': [m_ml.rho_, m_ml.sigma_],
    }, index=['rho', 'sigma']).round(4)
```

```
Out[42]:
```

	true	two-step	MLE
rho	0.6	0.5314	0.5523
sigma	1.0	0.9529	0.9586

Шесть видов прогноза

Хекит-модель допускает шесть содержательно различных прогнозов, доступных через один `predict()`. Каждой величине соответствует однобуквенный код; их можно комбинировать в произвольном порядке. Здесь $\lambda(t) = \phi(t)/\Phi(t)$ — обратный коэффициент Миллса.

Уравнение отбора (используется Z):

- **s** — вероятность включения: $P(S = 1 \mid Z) = \Phi(Z'\hat{\gamma})$
- **n** — вероятность невключения: $P(S = 0 \mid Z) = 1 - \Phi(Z'\hat{\gamma})$
- **p** — латентная склонность к включению: $Z'\hat{\gamma}$

Уравнение исхода (используется X , иногда оба):

- **o** — $\mathbb{E}[Y \mid X, Z, S = 1] = X'\hat{\beta} + \hat{\rho}\hat{\sigma}\lambda(Z'\hat{\gamma})$ — условие наблюдения
- **h** — $\mathbb{E}[Y^* \mid X] = X'\hat{\beta}$ — латентное безусловное

- $\mathbf{u} - \mathbb{E}[Y^* \mid X, Z, S = 0] = X'\hat{\beta} - \hat{\rho}\hat{\sigma}\phi(Z'\hat{\gamma})/[1 - \Phi(Z'\hat{\gamma})]$ — условие ненаблюдения

Вызов `predict(X=..., Z=...)` без `kind` возвращает все шесть колонок DataFrame в порядке `s, n, p, o, h, u`. Подмножество задаётся буквенной строкой: `kind='sn'`, `kind='ohu'` и т. д. Один символ возвращает одномерный массив. Длинные имена `'outcome'`, `'selection_prob'`, `'conditional'` сохранены для обратной совместимости и эквивалентны `'h'`, `'s'` и `'o'`.

```
In [43]: preds = m_ml.predict(X=X[:6], Z=Z[:6]) # default = all six
preds.assign(observed_y=y[:6], selected=S[:6]).round(3)
```

```
Out[43]:
```

	prob_selected	prob_not_selected	propensity	observed	hidden	unobserved	obs
0	0.626	0.374	0.322	1.405	1.084	0.548	
1	0.704	0.296	0.536	0.488	0.229	-0.390	
2	0.839	0.161	0.988	1.453	1.298	0.495	
3	0.499	0.501	-0.003	1.222	0.798	0.377	
4	0.363	0.637	-0.351	0.166	-0.381	-0.693	
5	0.071	0.929	-1.467	1.474	0.462	0.385	

Подмножество задаётся буквенной строкой. Сумма двух вероятностей равна единице; три $E[Y]$ -колонки связаны соотношением $\mathbb{E}[Y \cdot S] = \Phi(Z'\gamma) \cdot o + [1 - \Phi(Z'\gamma)] \cdot 0$ — то есть просто $\Phi(Z'\gamma) \cdot o$ для отобранных и 0 для остальных, если для последних приписать $Y = 0$.

```
In [44]: m_ml.predict(Z=Z[:6], kind='sn').round(3) # only the two probabilities
```

```
Out[44]:
```

	prob_selected	prob_not_selected
0	0.626	0.374
1	0.704	0.296
2	0.839	0.161
3	0.499	0.501
4	0.363	0.637
5	0.071	0.929

```
In [45]: m_ml.predict(X=X[:6], Z=Z[:6], kind='ohu').round(3) # the three E[Y] variants
```

```
Out [45]:
```

	observed	hidden	unobserved
0	1.405	1.084	0.548
1	0.488	0.229	-0.390
2	1.453	1.298	0.495
3	1.222	0.798	0.377
4	0.166	-0.381	-0.693
5	1.474	0.462	0.385

```
In [46]: m_ml.predict(X=X[:6], kind='h').round(3) # single letter -> 1-D ndarray
```

```
Out [46]: array([ 1.084,  0.229,  1.298,  0.798, -0.381,  0.462])
```

Та же оценка через формулу в стиле patsy

Вместо того чтобы собирать y , X , Z вручную, можно воспользоваться конструктором `from_formula(...)` в стиле statsmodels. У Хекита их две — отдельно для уравнения исхода и для уравнения отбора, с общим датафреймом:

```
In [47]: df = pd.DataFrame({
    'shared': shared, 'x_only': x_only, 'z_only': z_only,
    'inlf': S, 'lwage': y_full,
})
m_form = HeckitRegression.from_formula(
    outcome = 'lwage ~ 1 + shared + x_only',
    selection = 'inlf ~ 1 + shared + z_only',
    data=df, method='mle',
).fit()
print(pd.DataFrame({
    'beta (explicit)': m_ml.coef_,
    'beta (formula)': m_form.coef_,
}, index=['const', 'shared', 'x_only']).round(6))
```

	beta (explicit)	beta (formula)
const	1.010834	1.010834
shared	0.492478	0.492478
x_only	-0.301747	-0.301747

Бутстрап стандартных ошибок

Стандартные ошибки второго шага двухшаговой процедуры наивные: они игнорируют дисперсию $\hat{\gamma}$ с probit-шага. Парный бутстрап даёт корректные SE. Стандартные ошибки MLE, полученные из наблюдаемой гесссианы, уже корректны.

```
In [48]: boot = m_two.bootstrap(y, X, Z, n_boot=80, seed=0)
pd.DataFrame({
    'beta': m_two.coef_,
    'naive SE': m_two.bse_,
    'bootstrap SE': boot['beta_se'],
}, index=['const', 'shared', 'x_only']).round(4)
```

Out [48]:

	beta	naive SE	bootstrap SE
const	1.0267	0.0491	0.0479
shared	0.4903	0.0223	0.0214
x_only	-0.3010	0.0209	0.0205

Предельные эффекты

У модели Хекмана четыре стандартных типа предельных эффектов (Greene, *Econometric Analysis*, §19.5):

kind	Дифференцирует
'latent'	$E[Y^* X] = X'\beta$ — только X-сторона
'conditional'	$E[Y X, Z, S = 1]$ $= X'\beta + \rho\sigma$ $\lambda(Z'\gamma)$
'unconditional'	$E[Y \cdot S X, Z]$ $= \Phi(Z'\gamma)X'\beta$ $+ \rho\sigma \phi(Z'\gamma)$
'prob-selected'	$P(S = 1 Z)$ $= \Phi(Z'\gamma)$ — только Z-сторона

Если переменная входит в оба уравнения (здесь — **shared**), предельный эффект складывается из X-части и Z-части. Стандартные ошибки вычисляются дельта-методом по совместной ковариации MLE; для двухшаговой оценки они недоступны (**NaN**), так как совместной ковариации в замкнутой форме нет — нужен бутстрап.

Замкнутые формы построчных производных, где $\lambda = \phi/\Phi$ и $\delta(t) = \lambda(t)(t + \lambda(t))$, так что $d\lambda/dt = -\delta$:

$$\frac{\partial E[Y | S = 1]}{\partial v} = \beta_v \mathbf{1}\{v \in X\} - \gamma_v \rho \sigma \delta(Z'\gamma) \mathbf{1}\{v \in Z\}.$$

Передавая **feature_names_outcome** и **feature_names_selection**, мы указываем, что переменные с одинаковыми именами в X и Z — это одна и та же переменная.

```
In [49]: m_me = HeckitRegression(method='mle').fit(
    y, X, Z,
    feature_names_outcome=['shared', 'x_only'],
    feature_names_selection=['shared', 'z_only'],
)
```

```
ame_cond = m_me.ame(kind='conditional')
print(ame_cond.summary())
```

Heckit Regression Marginal Effects						
Dep. Variable:	y					
Method:	dydx					
At:	overall					
	dy/dx	std err	z	P> z	[0.025	0.975]
shared	0.4086	0.020	20.771	0.000	0.370	0.447
x_only	-0.3017	0.019	-15.738	0.000	-0.339	-0.264
z_only	-0.2017	0.021	-9.568	0.000	-0.243	-0.160

Все четыре вида предельных эффектов сведены в одну таблицу. Каждая строка усреднена по выборке (AME); пропуск означает, что переменная не входит в это уравнение.

```
In [50]: kinds = ['latent', 'conditional', 'unconditional', 'prob-selected']
tbl = pd.DataFrame(index=sorted({n for k in kinds for n in m_me.ame(kind=k).names}))
for k in kinds:
    me = m_me.ame(kind=k)
    tbl[k] = pd.Series(me.margeff, index=me.names)
tbl.round(4)
```

```
Out [50]:
```

	latent	conditional	unconditional	prob-selected
shared	0.4925	0.4086	0.3349	0.0859
x_only	-0.3017	-0.3017	-0.1527	NaN
z_only	NaN	-0.2017	0.2057	0.2064

Читая по столбцам:

- в **latent** стоят сами наклоны β (проверка корректности);
- **conditional** отличается от **latent** на **shared** (добавлена поправка со стороны Z) и совпадает с ним на **x_only** (X-only переменная, поправки нет);
- **unconditional** равномерно уменьшен: в $E[Y \cdot S]$ X-часть взвешивается вероятностью отбора $\Phi(Z'\gamma)$ вместо единицы;
- **prob-selected** содержит только Z-сторонние переменные с масштабом $\phi(Z'\gamma)$.

Реальные данные: Mroz (1987)

Каноническое применение Хекита — уравнение зарплаты замужних женщин из учебника Wooldridge. В наборе Mroz 753 наблюдения: 428 женщин участвуют в рабочей силе (`inlf = 1`) и имеют известную `lwage`; остальные 325 не работают, и `lwage` у них отсутствует. OLS на 428 работающих отвечает на условный вопрос — *чем определяется зарплата среди тех, кто решил работать?*, а

латентный вопрос — *какую зарплату получали бы женщины, если бы работали все?*
— это как раз то, что оценивает Хекит.

По Wooldridge (2010, пример 17.5):

- **Уравнение исхода:** `lwage ~ educ + exper + expersq`
- **Уравнение отбора:** `inlf ~ educ + exper + expersq + nwifeinc + age + kidslt6 + kidsge6`

Переменные `nwifeinc`, `age`, `kidslt6`, `kidsge6` играют роль exclusion restriction: они влияют на *решение работать*, но прямо не должны влиять на зарплату.

```
In [51]: try:
import wooldridge as woo
mroz = woo.data('mroz')
except ImportError:
    raise ImportError('Install with: pip install wooldridge')

print(f'shape: {mroz.shape}')
print(f'in labor force (inlf=1): {(mroz["inlf"]==1).sum()}')
print(f'out of labor force: {(mroz["inlf"]==0).sum()}')
print(f'lwage missing where inlf=0: {mroz.loc[mroz["inlf"]==0, "lwage"].isna().sum()}/'
      f'{(mroz["inlf"]==0).sum()} (should be 100% – the canonical selection setup)')
```

```
shape: (753, 22)
in labor force (inlf=1): 428
out of labor force:      325
lwage missing where inlf=0: 325/325 (should be 100% – the canonical selection setup)
```

Шаг 1 — наивный OLS на работающих. Так выглядит ответ, если игнорировать проблему отбора. Ниже мы сравним коэффициент при `educ` с Хекит-скорректированной оценкой.

```
In [52]: import statsmodels.formula.api as smf

ols = smf.ols('lwage ~ educ + exper + expersq',
              data=mroz[mroz['inlf'] == 1]).fit()
print(ols.summary().tables[1])
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.5220	0.199	-2.628	0.009	-0.912	-0.132
educ	0.1075	0.014	7.598	0.000	0.080	0.135
exper	0.0416	0.013	3.155	0.002	0.016	0.067
expersq	-0.0008	0.000	-2.063	0.040	-0.002	-3.82e-05

Шаг 2 — двухшаговый Хекит. В уравнение отбора входят exclusion-переменные; на втором шаге к уравнению зарплаты добавляется обратный коэффициент Миллса.

```
In [53]: from censtrunc import HeckitRegression

heck_ts = HeckitRegression.from_formula(
    outcome = 'lwage ~ educ + exper + expersq',
    selection = 'inlf ~ educ + exper + expersq + nwifeinc + age + kidslt6 + kidsge6',
    data=mroz, method='twostep',
```



```
.fit()
print(heck_ts.summary())
```

Heckman Selection Regression						
Method:	twostep		No. Observations:		753	
Selected (S=1):	428		Censored (S=0):		325	
sigma_e:	0.663629		rho:		0.048614	
Outcome equation (Y* = X * beta + e)						
	coef	std err	z	P> z	[0.025	0.975]
const	-0.5781	0.3051	-1.8948	0.0581	-1.1761	0.0199
educ	0.1091	0.0155	7.0243	0.0000	0.0786	0.1395
exper	0.0439	0.0163	2.6980	0.0070	0.0120	0.0758
expersq	-0.0009	0.0004	-1.9567	0.0504	-0.0017	0.0000
Selection equation (S* = Z * gamma + u, var(u) = 1)						
	coef	std err	z	P> z	[0.025	0.975]
const	0.2701	0.5083	0.5313	0.5952	-0.7262	1.2663
educ	0.1309	0.0252	5.1845	0.0000	0.0814	0.1804
exper	0.1233	0.0187	6.5906	0.0000	0.0867	0.1600
expersq	-0.0019	0.0006	-3.1454	0.0017	-0.0031	-0.0007
nwifeinc	-0.0120	0.0048	-2.4843	0.0130	-0.0215	-0.0025
age	-0.0529	0.0085	-6.2368	0.0000	-0.0695	-0.0362
kidslt6	-0.8683	0.1185	-7.3269	0.0000	-1.1006	-0.6360
kidsge6	0.0360	0.0435	0.8282	0.4075	-0.0492	0.1212

Note: two-step second-stage standard errors are naive; call `.bootstrap(y, X, Z)` for proper Heckman-correc

Шаг 3 — совместная MLE (асимптотически эффективна).

```
In [54]: heck_ml = HeckitRegression.from_formula(
outcome = 'lwage ~ educ + exper + expersq',
selection = 'inlf ~ educ + exper + expersq + nwifeinc + age + kidslt6 + kidsge6',
data=mroz, method='mle',
).fit()
print(heck_ml.summary())
```

Heckman Selection Regression						
Method:	mle	No. Observations:				753
Selected (S=1):	428	Censored (S=0):				325
sigma_e:	0.663631	rho:				0.048580
Log-Likelihood:	-832.8972					
Outcome equation (Y* = X * beta + e)						
	coef	std err	z	P> z	[0.025	0.975]
const	-0.5781	0.2577	-2.2432	0.0249	-1.0832	-0.0730
educ	0.1091	0.0148	7.3617	0.0000	0.0800	0.1381
exper	0.0439	0.0148	2.9622	0.0031	0.0148	0.0729
expersq	-0.0009	0.0004	-2.0622	0.0392	-0.0017	-0.0000
Selection equation (S* = Z * gamma + u, var(u) = 1)						
	coef	std err	z	P> z	[0.025	0.975]
const	0.2701	0.5089	0.5307	0.5956	-0.7274	1.2675
educ	0.1310	0.0254	5.1611	0.0000	0.0812	0.1807
exper	0.1234	0.0187	6.5870	0.0000	0.0867	0.1601
expersq	-0.0019	0.0006	-3.1463	0.0017	-0.0031	-0.0007
nwifeinc	-0.0122	0.0049	-2.4995	0.0124	-0.0217	-0.0026
age	-0.0528	0.0085	-6.2247	0.0000	-0.0694	-0.0362
kidslt6	-0.8683	0.1187	-7.3136	0.0000	-1.1010	-0.6356
kidsge6	0.0360	0.0435	0.8283	0.4075	-0.0492	0.1212

Сравнение коэффициентов

Сведём три оценки наклонов уравнения зарплаты. В учебнике Wooldridge отдача от образования по OLS близка к 0.108; скорректированная Хекитом отдача — около того же значения. Оценённая корреляция $\hat{\rho}$ между ошибкой уравнения зарплаты и ошибкой участия характеризует силу селекционного канала.

```
In [55]: import numpy as np
comparison = pd.DataFrame({
    'OLS (selected)': np.asarray(ols.params),
    'Heckit two-step': heck_ts.coef_,
    'Heckit MLE':      heck_ml.coef_,
}, index=heck_ts.outcome_feature_names_)
comparison.round(4)
```

```
Out [55]:
```

	OLS (selected)	Heckit two-step	Heckit MLE
const	-0.5220	-0.5781	-0.5781
educ	0.1075	0.1091	0.1091
exper	0.0416	0.0439	0.0439
expersq	-0.0008	-0.0009	-0.0009

```
In [56]: pd.DataFrame({
    'two-step': [heck_ts.rho_, heck_ts.sigma_],
    'MLE':      [heck_ml.rho_, heck_ml.sigma_],
}, index=['rho', 'sigma']).round(4)
```

```
Out [56]:
```

	two-step	MLE
rho	0.0486	0.0486
sigma	0.6636	0.6636

Интерпретация $\hat{\rho}$: положительное (отрицательное) значение означает, что ненаблюдаемые факторы, повышающие зарплату, одновременно повышают (понижают) вероятность работать. Если $\hat{\rho}$ мал или статистически незначим (это покажет тест Вальда либо LR-тест на $\rho = 0$), поправка на отбор эмпирически не нужна — достаточно обычной OLS.

Предельные эффекты на Mroz

Для содержательной интерпретации обычно нужны предельные эффекты на *условную* зарплату $E[\log \text{wage} \mid \text{работает}]$ и на *вероятность участия* $P(\text{работает})$. Ниже эффекты на участие вычисляются по стандартной probit-производной $\gamma_v \phi(Z'\gamma)$, усреднённой по выборке.

```
In [57]: ame_cond = heck_ml.ame(kind='conditional')
ame_prob = heck_ml.ame(kind='prob-selected')
pd.concat([
    ame_cond.summary_frame()[['dy/dx', 'std err', 'P>|z|']].rename(
        columns={'dy/dx': 'cond E[logwage]', 'std err': 'SE', 'P>|z|': 'pval'}),
    ame_prob.summary_frame()[['dy/dx', 'std err', 'P>|z|']].rename(
```

```
columns={'dy/dx': 'P(in LF)', 'std err': 'SE_p', 'P>|z|': 'pval_p'}),
], axis=1).round(4)
```

Out [57]:

	cond E[lwage]	SE	pval	P(in LF)	SE_p	pval_p
educ	0.1067	0.0143	0.0000	0.0394	0.0073	0.0000
exper	0.0417	0.0131	0.0015	0.0371	0.0052	0.0000
expersq	-0.0008	0.0004	0.0361	-0.0006	0.0002	0.0013
nwifeinc	0.0002	0.0007	0.7418	-0.0037	0.0015	0.0116
age	0.0010	0.0028	0.7356	-0.0159	0.0024	0.0000
kidslt6	0.0156	0.0463	0.7352	-0.2611	0.0319	0.0000
kidsge6	-0.0006	0.0021	0.7535	0.0108	0.0131	0.4069

Строка `educ` интерпретируется так: дополнительный год обучения повышает ожидаемый *log-wage* среди работающих на величину из столбца `cond E[lwage]` и одновременно повышает вероятность участвовать в рабочей силе на величину из столбца `P(in LF)`.

Z-only переменные (`nwifeinc`, `age`, `kidslt6`, `kidsge6`) также присутствуют в столбце `cond E[lwage]`, но с малыми и статистически незначимыми значениями. Это **не** пустые ячейки: данные переменные влияют на условную зарплату, но только через поправку обратного коэффициента Миллса $-\gamma_v \rho \sigma \delta(Z'\gamma)$. На данных Mroz оценённая корреляция $\hat{\rho}$ близка к нулю (около 0.03), поэтому соответствующий канал численно слаб, и значения близки к нулю — что и ожидается при слабой селекционной коррекции.

4.3 Эмпирическая проверка корректности

Три независимые проверки Хекит-оценщика:

1. **Восстановление DGP** показано в §4.2 — на симулированных данных с известными истинными параметрами двухшаговая и MLE-оценки попадают в их окрестность.
2. **Кросс-валидация против независимых реализаций** — `py4etrics.Heckit` (только двухшаговая) и R- `sampleSelection::selection` (двухшаговая и совместная MLE) на тех же данных. Обе обязаны сойтись в одну точку с нашей реализацией с точностью оптимизатора.
3. **Проверка конечными разностями** — формулы производных в `_heckit_effects.py` сверяются с формулами прогнозов в `heckit.py`, которые написаны независимо.

Кросс-валидация против `py4etrics` и `R-sampleSelection`

Восстановления истинного DGP недостаточно: ошибочно реализованный оценщик мог бы выдавать правдоподобные числа и проходить эту проверку. Решающая проверка — воспроизвести независимую реализацию на тех же данных. Сравним с двумя пакетами:

- `py4etrics.Heckit` (Hasebe et al.) — Python-реализация двухшагового алгоритма Хекмана.
- `sampleSelection::selection` (Toomet & Henningsen, JSS 27(7), 2008) — стандартный R-пакет для Хекита, реализующий обе версии (двухшаговую и совместную MLE) на том же логарифме правдоподобия, что и у нас.

Обе проверки включены в тесты

(`tests/test_heckit.py::test_twostep_matches_py4etrics` и `tests/test_r_reference.py::test_heckit_mle_matches_R`); ячейки ниже воспроизводят их.

Если `sampleSelection` локально не установлен, на macOS Apple Silicon достаточно один раз выполнить `brew install nlopt cmake pkg-config`, а затем `install.packages('sampleSelection', dependencies = TRUE)`.

```
In [58]: # Compare with py4etrics (two-step only -- py4etrics MLE is a no-op).
import warnings
try:
    from py4etrics.heckit import Heckit as Py4Heckit
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        ref = Py4Heckit(y, sm.add_constant(X), sm.add_constant(Z)).fit(method='twostep')
        diff_beta = np.max(np.abs(m_two.coef_ - ref.params))
        diff_gamma = np.max(np.abs(m_two.gamma_ - ref.select_res.params))
        print(f'max |beta_censtrunc - beta_py4etrics| = {diff_beta:.2e}')
        print(f'max |gamma_censtrunc - gamma_py4etrics| = {diff_gamma:.2e}')
        print(f'|sigma_censtrunc - sigma_py4etrics| = {abs(m_two.sigma_ - np.sqrt(ref.var_reg)):.2e}')
        print(f'|rho_censtrunc - rho_py4etrics| = {abs(m_two.rho_ - ref.corr_eqnerrors):.2e}')
except ImportError:
    print('py4etrics not installed; pip install py4etrics to enable the comparison.')

max |beta_censtrunc - beta_py4etrics| = 1.88e-09
max |gamma_censtrunc - gamma_py4etrics| = 4.68e-09
|sigma_censtrunc - sigma_py4etrics| = 1.08e-09
|rho_censtrunc - rho_py4etrics| = 3.16e-09
```

Оба пакета используют одни и те же замкнутые формулы для двухшаговой оценки, поэтому остаточное расхождение порядка 10^{-6} — это вычислительные погрешности линейной алгебры в numpy и statsmodels.

```
In [59]: # Compare with R's sampleSelection::selection (joint MLE).
# The R script expects a CSV with columns (y, s, x1, x2, z1, z2) and fits
# outcome 'y ~ x1 + x2' / selection 's ~ x1 + x2 + z1 + z2', so we build
# a fresh DGP with that exact layout.
import shutil, subprocess, json, tempfile, os
```

```

from pathlib import Path

def _find_r_heckit():
    for c in [Path('tests/reference/fit_heckit.R'),
              Path('../tests/reference/fit_heckit.R')]:
        if c.exists():
            return c
    return None

rscript, rpath = shutil.which('Rscript'), _find_r_heckit()
if rscript and rpath:
    rng_r = np.random.default_rng(20250601)
    n_r = 4000
    x1r = rng_r.normal(size=n_r); x2r = rng_r.normal(size=n_r)
    z1r = rng_r.normal(size=n_r); z2r = rng_r.normal(size=n_r)
    bb = np.array([0.5, 1.0, -0.4])
    gg = np.array([0.1, 0.3, -0.2, 0.5, 0.7])
    rho_r, sigma_r = 0.5, 1.2
    Sig = np.array([[sigma_r**2, rho_r*sigma_r], [rho_r*sigma_r, 1.0]])
    er, ur = rng_r.multivariate_normal([0, 0], Sig, size=n_r).T
    y_lat = bb[0] + bb[1]*x1r + bb[2]*x2r + er
    s_lat = gg[0] + gg[1]*x1r + gg[2]*x2r + gg[3]*z1r + gg[4]*z2r + ur
    s_obs = (s_lat > 0).astype(int)
    y_obs_r = np.where(s_obs == 1, y_lat, np.nan)

    Xr = np.column_stack([x1r, x2r])
    Zr = np.column_stack([x1r, x2r, z1r, z2r])
    m_ml_r = HeckitRegression(method='mle').fit(y_obs_r, Xr, Zr)

    fd, csvp = tempfile.mkstemp(suffix='.csv'); os.close(fd)
    pd.DataFrame({
        'y': y_obs_r, 's': s_obs,
        'x1': x1r, 'x2': x2r, 'z1': z1r, 'z2': z2r,
    }).to_csv(csvp, index=False, na_rep='NA')
    try:
        out = subprocess.run([rscript, str(rpath), csvp],
                              capture_output=True, text=True, timeout=180, check=True)
        rdat = json.loads(out.stdout)['mle']
        r_beta = np.array([rdat['outcome']['(Intercept)'],
                          rdat['outcome']['x1'], rdat['outcome']['x2']])
        r_gamma = np.array([rdat['selection']['(Intercept)'],
                           rdat['selection']['x1'], rdat['selection']['x2'],
                           rdat['selection']['z1'], rdat['selection']['z2']])
        print(f"max |beta_censtrunc - beta_R| = {np.max(np.abs(m_ml_r.coef_ - r_beta)):.2e}")
        print(f"max |gamma_censtrunc - gamma_R| = {np.max(np.abs(m_ml_r.gamma_ - r_gamma)):.2e}")
        print(f"|sigma_censtrunc - sigma_R| = {abs(m_ml_r.sigma_ - rdat['sigma']):.2e}")
        print(f"|rho_censtrunc - rho_R| = {abs(m_ml_r.rho_ - rdat['rho']):.2e}")
        print(f"|logLik_censtrunc - logLik_R| = {abs(m_ml_r.llf_ - rdat['logLik']):.2e}")
    except Exception as exc:
        print(f'R run failed: {exc!r}')
    finally:
        os.remove(csvp)
else:
    print('Rscript not available at build time; see tests/test_r_reference.py for the gated to

```

```

max |beta_censtrunc - beta_R| = 5.68e-06
max |gamma_censtrunc - gamma_R| = 6.81e-06
|sigma_censtrunc - sigma_R| = 1.62e-06
|rho_censtrunc - rho_R| = 4.53e-06
|logLik_censtrunc - logLik_R| = 9.93e-08

```

Два независимых оптимизатора (scipy `L-BFGS-B` и R- `maxLik`) работают на одном и том же совместном логарифме правдоподобия: оценки параметров, σ , ρ и сам логарифм совпадают с точностью оптимизатора (порядка 10^{-3}).

Проверка конечными разностями `predict` . Формулы производных в `_heckit_effects.py` и код прогнозов в `heckit.py` реализованы независимо, поэтому центральная конечная разность от `predict()` — это независимая

проверка аналитической формулы для dy/dx . Ниже вычисления выполняются в точке $X = \bar{X}$, $Z = \bar{Z}$ с возмущением каждого регрессора.

```
In [60]: h = 1e-4
X_mean = X.mean(axis=0, keepdims=True)
Z_mean = Z.mean(axis=0, keepdims=True)
def cond_at(Xm, Zm):
    return float(m_me.predict(X=Xm, Z=Zm, kind='conditional').mean())

mem = m_me.mem(kind='conditional')
rows = []
for j, name in enumerate(['shared', 'x_only']):
    Xp, Xm_ = X_mean.copy(), X_mean.copy()
    Xp[0, j] += h; Xm_[0, j] -= h
    Zp, Zm_ = Z_mean.copy(), Z_mean.copy()
    if name == 'shared':
        Zp[0, 0] += h; Zm_[0, 0] -= h
    num = (cond_at(Xp, Zp) - cond_at(Xm_, Zm_)) / (2*h)
    rows.append((name, mem.margeff[mem.names.index(name)], num))
# z_only is Z-only.
Zp, Zm_ = Z_mean.copy(), Z_mean.copy()
Zp[0, 1] += h; Zm_[0, 1] -= h
num_z = (cond_at(X_mean, Zp) - cond_at(X_mean, Zm_)) / (2*h)
rows.append(('z_only', mem.margeff[mem.names.index('z_only')], num_z))
pd.DataFrame(rows, columns=['variable', 'analytic dy/dx', 'finite-difference']).round(8)
```

Out [60]:

	variable	analytic dy/dx	finite-difference
0	shared	0.405509	0.405509
1	x_only	-0.301747	-0.301747
2	z_only	-0.208959	-0.208959

Столбцы совпадают до семи-восьми знаков: аналитические формулы производных в `_heckit_effects.py` согласованы с формулами прогнозов в `heckit.py`.